

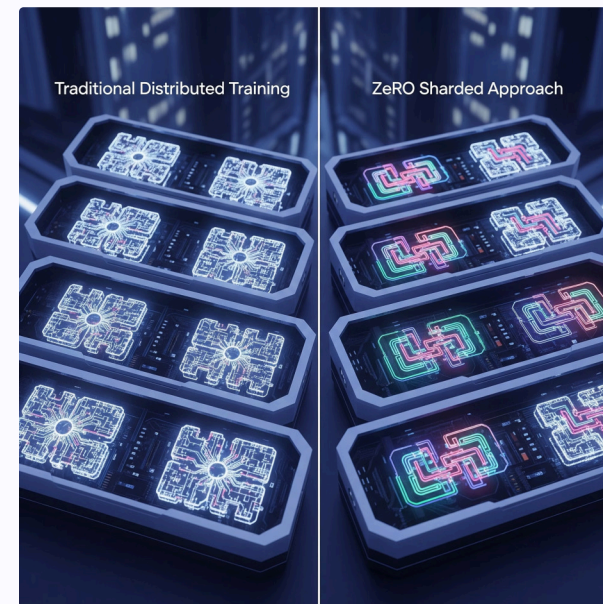


DeepSpeed ZeRO-2/3: Memory Sharding & Throughput Tricks for Giant Models

Practical strategies for training billion-parameter models efficiently across multiple GPUs

Why ZeRO?

- Billion+ parameter models **exceed single-GPU memory capacity**
- Naive data parallel training **duplicates states $N\times$** across ranks
- ZeRO **shards states** across data-parallel workers
- Result: fit larger models, larger batches, lower training cost



Recap: ZeRO Stages

Stage 1

Shard **optimizer states** only

Adam states typically consume 2×
model size

Stage 2

Shard **optimizer states** +
gradients

Gradients consume 1× model size

Stage 3

Shard **optimizer states** +
gradients + **parameters**

Full sharding eliminates all
redundancy

Each stage removes more redundancy → provides more memory headroom for training

What ZeRO-2 Adds

Gradient partitioning & reduce-scatter

Eliminates the need to keep full gradients during backward pass

Bucketing + overlapped communication

Hides communication latency behind computation

Typical gains: 2–3× larger batch size

Compared to Stage 1, with minimal throughput impact



Good balance of complexity vs.
benefits

What ZeRO-3 Adds

Parameters are fully sharded across GPUs

Just-in-time gathering of parameters per layer during forward/backward

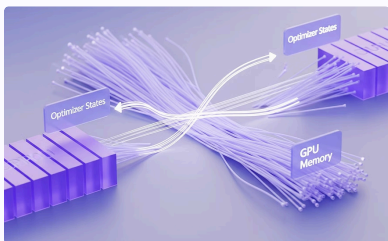
True memory scaling with number of GPUs (nearly linear)

Enables **>20B parameter models** on modest GPU hardware



Requires careful communication/compute overlap to avoid pipeline stalls

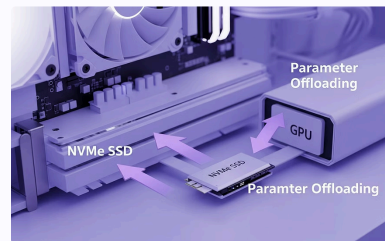
Offloading Options



ZeRO-Offload

Move optimizer states and gradients to **CPU memory** (pinned)

Lower overhead, good for systems with ample CPU RAM



ZeRO-Infinity

Offload **parameters & activations** to **CPU/NVMe**

Highest capacity, but I/O bound

Trade-off: Memory capacity $\uparrow$, Latency/I/O demands $\uparrow \rightarrow$ Tune settings based on PCIe/NVMe speeds

Typical DeepSpeed Config (Conceptual)

```
{
  "train_micro_batch_size_per_gpu": 4,
  "gradient_accumulation_steps": 8,
  "zero_optimization": {
    "stage": 3,
    "overlap_comm": true,
    "contiguous_gradients": true,
    "reduce_bucket_size": 5e8,
    "stage3_prefetch_bucket_size": 5e8,
    "stage3_param_persistence_threshold": 1e5,
    "offload_optimizer": {"device": "cpu", "pin_memory": true}
  },
  "bf16": {"enabled": true},
  "activation_checkpointing": {"partition_activations": true}
}
```

Tune bucket sizes to your interconnect & model architecture

Higher values may improve throughput on fast networks (InfiniBand HDR+)

Performance Levers



Communication Overlap

Enable `overlap_comm`; size buckets to saturate interconnects

Adjust bucket sizes based on network bandwidth (larger for faster networks)



Mixed Precision

Use `bf16/fp16` + fused kernels (bias+GELU, RMSNorm)

BF16 offers better numerical stability than FP16 for large models



Gradient Accumulation

Accumulate gradients to hit stable global batch size

Balance between memory usage and convergence requirements



Activation Checkpointing

Trade compute for memory by recomputing activations

Essential for very deep models with long sequence lengths



Practical Tuning Flow



Staged Approach

Start **Stage 1** → **2** → **3**

Verify stability at each step



Batch Size Tuning

Dial **micro-batch** up to near-OOM

Step down 10-15% for stability buffer



Communication Optimization

Sweep **bucket sizes** & **prefetch**

Optimize for tokens/sec throughput



Consider Offloading

Add **CPU/NVMe offload**

Only if capacity is still tight

Monitoring & Debugging

Key Metrics to Watch

tokens/sec/GPU
throughput

GPU memory utilization
patterns

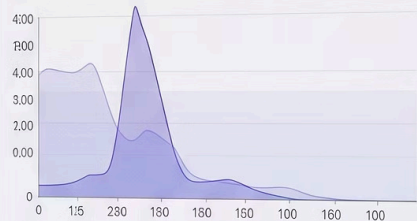
NCCL wait time in
profiler

Common Issues & Fixes

- If JIT-gathers stall → reduce param persistence threshold, prefetch earlier
- If CPU offload slow → check memory pinning, NUMA configuration, PCIe generation, disk IOPS
- Divergence in training? Test loss scaling, disable suspect kernel fusions

Distributed Training spanning multiple nodes

GPU Utilization



Network Usage

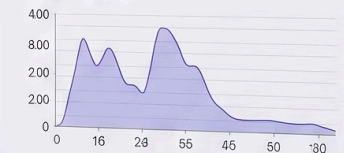
Utilization
GPU Stream gpus

Memory Usage



Seen Traffic

Network Traffic



When to Choose What

<10B Parameters

ZeRO-2 often sufficient

- Good balance of memory savings and communication overhead
- Higher throughput than ZeRO-3

10-40B Parameters

ZeRO-3 w/ bf16 + checkpointing

- Full parameter sharding becomes necessary
- BF16 provides numerical stability

40B+ or Limited GPUs

ZeRO-3 + Offload/Infinity

- CPU memory and NVMe become essential
- Accept lower throughput for capacity



Network-constrained environments: Prefer larger bucket sizes, fewer parameter gathers

Key Takeaways



Memory Efficiency

ZeRO-2/3 remove N-way redundancy → **huge memory savings**



Performance Optimization

Overlap + smart bucketing = **real throughput gains**



Extended Capacity

Offload extends capacity but **must be I/O-tuned**



Iterative Tuning

Measure, iterate, and right-size configs to hardware

